Step Cards — AI Prompts

AI Prompts — Step Cards

Team Kaleidoscope | Author: Naman + Dipesh | Date: Nov 2, 2026 Export this file to PDF and attach to the Airtable submission form under "AI Prompts PDF".

OVERVIEW

Step Cards uses **three AI calls** to turn a teacher's spoken instruction into per-student picture step cards. Every call runs **Groq Qwen3-32B** via the OpenAI-compatible Chat Completions API. No student names, photos, or diagnoses are ever sent to the model — only the instruction text.

#	Call	Purpose	Where it lives
1	`CLASSROOM_PROMPT`	Break a spoken instruction into numbered steps, auto-split long ones into multi-part lessons.	`src/lib/lessons.functions.ts`
2	`REWRITE_PROMPT`	Rewrite each step at a student's reading level (grade2 / grade4 / pictogram_only).	`src/lib/lessons.functions.ts`
3	ARASAAC enrichment	Match each step's `icon_keyword` to an ARASAAC pictogram via free public API.	`src/lib/step-cards/arasaac.ts`

MODEL + TRANSPORT CONFIG

File: `src/lib/ai.server.ts`

```
const GROQ_BASE = "https://api.groq.com/openai/v1";
const MODEL = "qwen/qwen3.8-27b";
```

```
const provider = createOpenAI({
  baseURL: GROQ_BASE,
  apiKey,
  compatibility: "compatible",   // forces /v1/chat/completions, Groq does not
        support /v1/responses
});

const { text } = await generateText({
  model: provider(MODEL),
  system: instructions +
    "\n\nRespond with ONLY a valid JSON object. No markdown, no explanation, no code
        blocks — raw JSON only.",
  prompt: "/no_think " + input,  // disables Qwen3 chain-of-thought reasoning for
        latency
  maxTokens: 1024,
});
```

Why `/no_think`: Qwen3 emits `<think>…</think>` chain-of-thought by default; the prefix disables it and brings per-call latency to ~1–2 seconds — short enough to live inside the teacher's attention span.

Response hardening: We strip `<think>` tags and markdown fences the model occasionally adds, then regex-extract the outer `{…}` and `JSON.parse` it. The parsed object is validated against a Zod schema before any UI code sees it. Invalid shapes raise a user-facing error ("Couldn't turn that into steps. Try saying it a different way.") rather than crashing.

CALL 1 — CLASSROOM_PROMPT (instruction → structured lesson)

File: `src/lib/lessons.functions.ts` (verbatim)

```
 You are an assistive technology system helping a special education teacher
support autistic and ADHD students in a classroom. The teacher has just spoken
an instruction aloud.

Break the instruction into numbered, plain-language steps a student can follow
one action at a time.

Research-based cap: a routine of cards MUST be 2 to 6 steps. More than 6 cards
overloads working memory for the students we serve and defeats the fading
practice we rely on. If the full instruction clearly has more than 6 real
actions, split it into separate ROUTINE PARTS — each part a distinct short
routine of 2–6 steps that can stand on its own (e.g. "Set up station" /
"Make sandwich" / "Clean up").
```

```
OUTPUT (JSON ONLY):
{
  "title": "<overall short title, max 6 words, sentence case>",
  "parts": [
    { "title": "<part title, max 6 words>", "steps": [
        { "n": 1, "text": "…", "icon_keyword": "…", "timer_seconds": null },
        …
    ] },
    …
  ]
}

RULES:
1.  parts: 1 part for most instructions. Add more parts (up to 4) ONLY when the
    instruction truly spans separate stages that each have their own goal.
    Never split just to show off — short instructions stay as 1 part.
2.  Each part has 2 to 6 steps. Never more than 6. Steps number from 1 inside
    each part.
3.  Each step is ONE physical action. Never combine two actions in one step.
4.  Start each step with a specific action verb: Wash, Brush, Rinse, Pour,
    Measure, Pick up, Put, Mix, Wait. NOT vague verbs like "Get" or "Do".
5.  Plain language: short words, simple sentences. Maximum 8 words per step.
6.  Concrete nouns only: "the soap" not "the item". "your toothbrush" not "it".
7.  Numbers as numerals: "2 cups" not "two cups".
8.  Only add a "what to do when finished" step (e.g. "Return to your seat") if
    the instruction implies the student has left their place and needs to
    return. Do NOT add it for self-contained actions.
9.  Do NOT add praise, safety warnings, or steps not implied by the instruction.
10. icon_keyword: the action of THAT step as "verb object" in 1–2 plain English
    words, used to find a picture in the ARASAAC symbol library
    (e.g. "Rinse your hands" → "rinse hands", "Coil the cable" → "cable",
     "Sit down at your desk" → "sit down").
    Use the object alone when the verb is vague (put, get, take).
    Never abstract words or numbers.
11. No idioms, metaphors or sarcasm. Active voice only.
12. timer_seconds: only when the teacher says or clearly implies how long a
    step lasts (e.g. "scrub for 20 seconds", "read quietly for 5 minutes"),
    give that length in seconds. Otherwise null.
13. Each part title names what that part accomplishes
    (e.g. "Measure ingredients", "Mix the dough", "Clean up").
    Different from the overall title.
```

Zod schema that validates the response:

```
const stepSchema = z.object({
  n: z.number().optional(),           // optional; we always override from index
  text: z.string(),
  icon_keyword: z.string(),
```

```
});
const baseStepSchema = stepSchema.extend({
  timer_seconds: z.number().nullable().optional(),
});
const partSchema = z.object({
  title: z.string(),
  steps: z.array(baseStepSchema),
});
const baseSchema = z.object({
  title: z.string(),
  steps: z.array(baseStepSchema).optional(),  // legacy single-part shape
  parts: z.array(partSchema).optional(),      // new multi-part shape
});
```

Post-processing guarantees: - Clamp to max 4 parts and max 6 steps per part (defense against prompt drift). - Renumber `n` from the array index (don't trust the model). - Reject `timer_seconds < 5` and clamp above 1800 (sanity). - Suffix multi-part titles: `"<base> · Part k of N: <part title>"` so students see which part they're on.

CALL 2 — REWRITE_PROMPT (grade-level personalization)

File: `src/lib/lessons.functions.ts` (verbatim)

```
You are an assistive technology system rewriting classroom steps for a
student's reading level.

READING LEVELS:
- grade2: 1-3 words per step. Verb + noun only. ("Wash hands", "Wait")
- grade4: 4-6 words. Simple sentences. ("Put your tray on the shelf")
- pictogram_only: 1-2 words maximum. Single noun or verb.

RULES:
1. Do NOT change what the step asks. Same action, different words.
2. Do NOT add or remove steps. Keep the same number and order of steps.
3. Do NOT add praise or encouragement.
4. Keep icon_keyword from the original unchanged.
```

How it's called (per part × reading level, in parallel):

```
const levels = [...new Set(students.map((s) => s.reading_level))]
  .filter((l) => l !== "grade6");       // grade6 uses the base steps as-is

const rewritePromises = parts.flatMap((p) =>
  levels.map((level) =>
```

```
    generateJson(
      rewriteSchema,
      REWRITE_PROMPT,
      JSON.stringify({
        reading_level: level,
        steps: p.steps.map(({ n, text, icon_keyword }) => ({ n, text, icon_keyword })),
      }),
    ).catch(() => null),  // one level failing never blocks the whole class
  ),
);
```

Zod schema:

```
const rewriteSchema = z.object({ steps: z.array(stepSchema) });
```

Why we parallelize per level, not per student: A class of 3 students with the same reading level needs one LLM call, not three. We dedupe the set first.

CALL 3 — ARASAAC PICTOGRAM ENRICHMENT

Not an LLM call — it's a lookup against the free ARASAAC public API, but it's part of the AI pipeline because `icon_keyword` is produced by Call 1 for exactly this purpose.

File: `src/lib/step-cards/arasaac.ts`

Flow per step: 1. Build a query list from `icon_keyword`: the keyword itself, then the first non-filler word of the step text, then individual words — up to 4 queries. 2. Hit `GET https://api.arasaac.org/v1/pictograms/en/search/{query}?limit=25` for each query. 3. Score each candidate against the step's words (exact match 100 → shared tokens 80 → verb-only 60 → single-word match 50 → prefix match 30 → nothing 0). 4. Prefer the highest-scoring match across all queries. Below a threshold, we show the step number instead of a wrong picture — a confusing icon is worse than no icon. 5. Fallback map: a hand-curated `FALLBACK_IDS` object in `src/lib/step-cards/data.ts` short-circuits the API for common classroom actions ("rinse hands", "pour water") so those always resolve, even offline.

Filler words removed before scoring:

```
 the, a, an, your, you, one's, my, his, her, their,
on, of, to, in, at, with, for, and, some
```

Vague verbs that disqualify the verb as a standalone query:

```
get, do, make, go, have, take, use, put, pick, up
```

License / attribution: ARASAAC symbols are released under CC BY-NC-SA 4.0 by Government of Aragón. Author: **Sergio Palao**. Attribution is baked into the app footer and the GitHub README.

FULL PIPELINE SEQUENCE (per teacher instruction)

```
Teacher speaks → Web Speech API → transcript (browser-only, nothing sent anywhere)
      │
      ▼
Call 1: Groq Qwen3-32B / CLASSROOM_PROMPT       (~1.5s)
      │  →  { title, parts: [{ title, steps[] }] }
      ▼
In parallel for each (part × reading level) and (part → pictograms):
   Call 2: Groq Qwen3-32B / REWRITE_PROMPT      (~1–2s per level)
   Call 3: ARASAAC search → pictogram URLs      (~200–600ms per step)
      │
      ▼
Teacher sees per-student card previews → reviews → taps "Approve"
      │
      ▼
Supabase realtime push → student device shows cards
```

Total end-to-end latency on a typical 5-step instruction with 2 reading levels: ~2.5–3.5 seconds. Fits inside the teacher's attention window.

SAFETY, PRIVACY, AND ATTRIBUTION

- **No PII to the model:** The only text sent to Groq is the instruction itself. No student names, class names, user IDs, or metadata.
- **No diagnoses:** Reading level is a student setting (grade2 / grade4 / grade6 / pictogram_only) — not a diagnosis label.
- **No lyrics or copyrighted text generation:** The prompts explicitly constrain the model to rewriting the teacher's own instruction. Nothing creative, nothing from training data.
- **Teacher approval is required** before any AI-generated card appears on a student's device. The approve step is a legal and ethical safety: a human in the loop for every output.
- **Graceful degradation:** If Call 2 fails for one reading level, that student falls back to the grade6 base steps. If Call 3 fails for one step, the step shows a number instead of a wrong picture.
- **ARASAAC attribution:** In-app footer and README both credit Sergio Palao / Government of Aragón (CC BY-NC-SA 4.0).

- **AI code attribution:** Every AI-assisted function in the repo carries an `// AI-generated — reviewed and tested by <name>` comment per challenge rules.

REPRODUCIBILITY

To reproduce any output locally:

```
git clone https://github.com/dgupta98/NueroChallange
cd step-cards
pnpm install
echo "GROQ_API_KEY=<your-key>" > .env
echo "VITE_SUPABASE_URL=…"     >> .env
echo "VITE_SUPABASE_ANON_KEY=…" >> .env
pnpm dev
```

Then open `http://localhost:3000`, create a class with 2–3 students at different reading levels, and type any instruction into the composer. The three prompts above will fire in sequence and the output will match the live app at https://nuero-challange.vercel.app/.